Project Report

Resume–Job Matching & Talent-Marketplace Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms - 3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by:
Vakkalanka Sai Bhaskara Sujit (2520090085)
Mahankali Sai Tarun (2520090020)

Team Number: 14

Section: 10

Under the guidance of:

K. Chandusha

Asst. Prof., CS&IT



KL (Deemed to be) University
Bachupally, Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled **"Resume–Job Matching & Talent-Marketplace Engine"** is a record of Bonafide work of **Vakkalanka Sai Bhaskara Sujit – 2520090085** and **Mahankali Sai Tarun – 2520090020** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology (Deemed to be) University. The results embodied in this report have not been copied from any other departments/University/Institute.

Vakkalanka Sai Bhaskara Sujit – 2520090085

Mahankali Sai Tarun – 2520090020

K L (Deemed to be) University

COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **"Resume–Job Matching & Talent-Marketplace Engine"** is a bonafide work done and submitted by **Vakkalanka Sai Bhaskara Sujit – 2520090085** and **Mahankali Sai Tarun – 2520090020** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology KL (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our guide, **K. Chandusha**, Asst. Prof., CS&IT, which cannot be expressed in words for her tremendous support, encouragement, and guidance throughout this project.

We express our gratitude to **Dr. P. Venkateshwar Rao** (CSE) **/ Dr. Khaja Shareef** (CSIT) **/ Dr. Sreenath Itikela** (AI&DS), Head of the Department for Computer Science and Engineering **/** Department of Computer Science and Information Technology **/** Department of Artificial Intelligence and Data Science for providing us with adequate facilities, ways, and means by which we were able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff, and all who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation during our work.

Vakkalanka Sai Bhaskara Sujit – 2520090085

Mahankali Sai Tarun – 2520090020

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature Survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software Requirements	10
6.	Implementation (6-Module DSA Coverage)	11
7.	Results & Discussion	13
8.	Conclusion & Future Scope	16
9.	References	17

1. ABSTRACT

Modern recruitment platforms process thousands of resumes and job postings, making accurate candidate matching increasingly difficult because of inconsistent skill terminology, diverse résumé formats, incomplete candidate information, and large hiring volumes. This project presents a **Résumé–Job Matching & Talent-Marketplace Engine** that efficiently indexes résumés and job descriptions, normalizes skill vocabularies into a unified representation, and evaluates candidate-to-job compatibility using weighted similarity scoring. The system further computes an optimal assignment of candidates to available roles while minimizing skill gaps and maximizing overall hiring efficiency. In addition, it identifies the minimum set of essential skills required to staff a project team effectively, supporting workforce planning, recruitment decisions, and resource allocation. By integrating efficient data structures, searching, hashing, graph-based matching, and optimization algorithms, the proposed solution provides a scalable, accurate, transparent, and reliable hiring platform that reduces manual screening effort, improves recruitment quality, enhances decision-making, and streamlines the overall talent acquisition process.

Team Number: Team 14

Team Members:

1. Vakkalanka Sai Bhaskara Sujit (2520090085)
2. Mahankali Sai Tarun (2520090020)

2. INTRODUCTION

The recruitment industry faces significant challenges in efficiently matching candidates with appropriate job roles due to the sheer volume of unstructured resumes and diverse job descriptions. The introduction of applicant tracking systems (ATS) has automated some of this process, but they often lack the algorithmic depth required to provide explainable, deterministic, and optimal candidate-to-job matches.

The **Talent Engine** addresses this by employing a deterministic candidate-to-role matching and workforce-staffing system built from scratch using pure Java SE 17. By avoiding "black-box" AI models, the engine ensures that all matching decisions are mathematically transparent and auditable. The system leverages six core algorithmic paradigms, including inverted indices for fast lookups, string matching algorithms (KMP, Rabin-Karp) for resume parsing, dynamic programming (Levenshtein) for skill normalization, graph algorithms (Hungarian, Edmonds-Karp) for optimal allocation, greedy approximations for team staffing, and randomized sorting for candidate ranking. With a visually rich editorial design, the project acts as a complete, scalable, and highly efficient talent marketplace.

3. LITERATURE SURVEY

Applicant Tracking Systems (ATS) have evolved significantly over the years. Early systems primarily relied on simple keyword matching using regular expressions or basic SQL queries to filter candidates. However, these systems struggled with typographical errors and synonymous skill representations (e.g., “JS” vs. “JavaScript”).

More recent platforms have integrated Machine Learning (ML) and Natural Language Processing (NLP) models to compute semantic similarities between resumes and job descriptions. While these ML models provide high flexibility, they often operate as “black boxes,” making it difficult for recruiters to understand exactly why a candidate was recommended or rejected. Furthermore, these AI models require massive computational resources and external dependencies.

In contrast, classical algorithmic approaches—such as using the Hungarian Algorithm for optimal assignment and Levenshtein distance for string normalization—offer deterministic, explainable, and highly efficient matching. Similar academic projects have explored using bipartite graphs for job assignment and TF-IDF (Term Frequency-Inverse Document Frequency) for resume ranking. By strictly using standard Data Structures and Algorithms, the Talent Engine provides a robust alternative to ML-based systems that guarantees exact and explainable results without relying on external frameworks.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement:

The inefficiency and lack of transparency in modern recruitment platforms lead to suboptimal candidate-role assignments and extensive manual screening.

Problem Description:

Large volumes of resumes containing inconsistent skill terminology and formats make accurate and optimal candidate-to-job matching computationally challenging. Existing black-box AI systems lack explainability, leaving recruiters unable to justify hiring decisions or efficiently optimize team staffing based on precise skill requirements.

Objectives:

- To build a highly efficient inverted index for $O(1)$ average-time skill lookups across resumes and job postings.
- To implement Knuth-Morris-Pratt (KMP) and Rabin-Karp substring search for deep resume text inspection without false positives.
- To normalize inconsistent skill inputs handling typos and abbreviations using Dynamic Programming (Levenshtein Distance).
- To compute mathematically optimal 1:1 candidate-to-job allocations that minimize overall skill gaps using Graph Algorithms (Hungarian Algorithm).
- To solve capacity-constrained role allocations and identify recruitment bottlenecks via Network Flow (Edmonds-Karp Min-Cut).
- To formulate and solve workforce staffing problems efficiently using exact Bitmask Dynamic Programming and Greedy Set Cover approximations.
- To utilize Randomized QuickSort and multi-threaded Java Parallel Streams for high-throughput ranking and analytics.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Multi-core Processor (Intel Core i5 / AMD Ryzen 5 or higher recommended for parallel analytics)
- Minimum 4GB RAM (8GB+ recommended for large candidate datasets)
- Minimum 100MB of free storage space

Software Requirements:

- Java Development Kit (JDK 17, 21, or 25 LTS)
- Modern Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari)
- Operating System: Windows, macOS, or Linux

6. IMPLEMENTATION (COMPLETE 6-MODULE DSA COVERAGE)

The Talent Engine is architected around six core data structure and algorithm modules:

Module 1: Inverted Index & Hash Lookup (`SkillIndexer.java`)

Uses `HashMap<String, Set<String>>` to index canonical skills to candidate and job IDs for $O(1)$ average time lookups across the marketplace.

```
public class SkillIndexer {
    private final Map<String, Set<String>> skillToCandidateIds = new HashMap<>();
    private final Map<String, Set<String>> skillToJobIds = new HashMap<>();

    public synchronized void indexCandidate(Candidate candidate) {
        if (candidate == null || candidate.getSkills() == null) return;
        String cid = candidate.getId();
        for (String skill : candidate.getSkills()) {
            String canonical = skill.trim();
            if (!canonical.isEmpty()) {
                skillToCandidateIds.computeIfAbsent(canonical, k -> new HashSet<>()).add(cid);
            }
        }
    }
}
```

Module 2: String Algorithms (`StringAlgorithms.java`)

Implements Knuth-Morris-Pratt (KMP) with π-table prefix function and Rabin-Karp polynomial rolling hash for deep candidate resume searching in $O(N+M)$ time.

```
// Computes the KMP Prefix Function (pi-table / failure table) in O(M) time
public static int[] computePrefixFunction(String pattern) {
    int m = pattern.length();
    int[] pi = new int[m];
    int j = 0;
    for (int i = 1; i < m; i++) {
        while (j > 0 && Character.toLowerCase(pattern.charAt(i)) != Character.toLowerCase(pattern.charAt(j))) {
            j = pi[j - 1];
        }
        if (Character.toLowerCase(pattern.charAt(i)) == Character.toLowerCase(pattern.charAt(j))) {
            j++;
        }
        pi[i] = j;
    }
    return pi;
}
```

Module 3: Advanced Dynamic Programming (`LevenshteinDistance.java` & `BitmaskTeamOptimizer.java`)

Computes matrix edit distance to normalize misspelled user inputs (e.g., `"jsavscript"` $\rightarrow$ `"javascript"`) in $O(M \times N)$, and solves minimal team staffing using Bitmask DP in $O(2^K \cdot N)$.

```
// Levenshtein Edit Distance for automatic skill normalization
public static int compute(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            int cost = (s1.charAt(i - 1) == s2.charAt(j - 1)) ? 0 : 1;
            dp[i][j] = Math.min(dp[i - 1][j] + 1, // Deletion
                       Math.min(dp[i][j - 1] + 1, // Insertion
                                dp[i - 1][j - 1] + cost)); // Substitution
        }
    }
    return dp[m][n];
}
```

Module 4: Network Flow & Optimal Assignment (`HungarianAlgorithm.java` & `NetworkFlowEngine.java`)

Precomputes a cost matrix based on skill gaps and executes the Kuhn-Munkres algorithm for globally optimal 1:1 matching in $O(N^3)$, alongside Edmonds-Karp Max-Flow with Min-Cut bottleneck analysis in $O(V \cdot E^2)$.

```
// Hungarian Algorithm: Cost matrix formulation minimizing total skill gap
double[][] costMatrix = new double[dim][dim];
for (int i = 0; i < dim; i++) {
    for (int j = 0; j < dim; j++) {
        if (i < numCandidates && j < numJobs) {
            MatchResult mr = WeightedMatcher.computeMatch(candidates.get(i), jobs.get(j));
            costMatrix[i][j] = 100.0 - mr.getTotalScore(); // Cost = skill gap
        } else {
            costMatrix[i][j] = 1000.0; // Dummy penalty for matrix squareness
        }
    }
}
// Kuhn-Munkres potential labelling and augmenting path traversal computes optimal assignment
```

Module 5: NP-Completeness & Team Optimization (`GreedySetCover.java`)

Approximates the NP-Hard set cover problem to find the smallest project team covering all required skills in polynomial $O(|U| \cdot N)$ time with proven $H(|U|) \leq \ln|U| + 1$ harmonic bound.

```
// Iterative selection of maximum marginal skill contributor
while (!uncovered.isEmpty()) {
    Candidate bestCandidate = null;
    List<String> bestNewlyCovered = new ArrayList<>();
    int maxContribution = 0;

    for (String cid : remainingPoolCandidateIds) {
        Candidate cand = candidateMap.get(cid);
        List<String> newlyCovered = new ArrayList<>();
        for (String skill : cand.getSkills()) {
            if (uncovered.contains(skill.trim())) newlyCovered.add(skill);
        }
        if (newlyCovered.size() > maxContribution) {
            maxContribution = newlyCovered.size();
            bestCandidate = cand;
            bestNewlyCovered = newlyCovered;
        }
    }
    if (maxContribution == 0) break; // Maximum achievable coverage reached
    team.add(bestCandidate);
    uncovered.removeAll(bestNewlyCovered);
}
```

Module 6: Randomized & Parallel Computing (`RandomizedAlgorithms.java` & `ParallelAnalytics.java`)

Implements Randomized QuickSort with random pivot selection for $O(N \log N)$ expected ranking time, along with multi-threaded Java `ForkJoinPool` parallel streams for multi-core analytics.

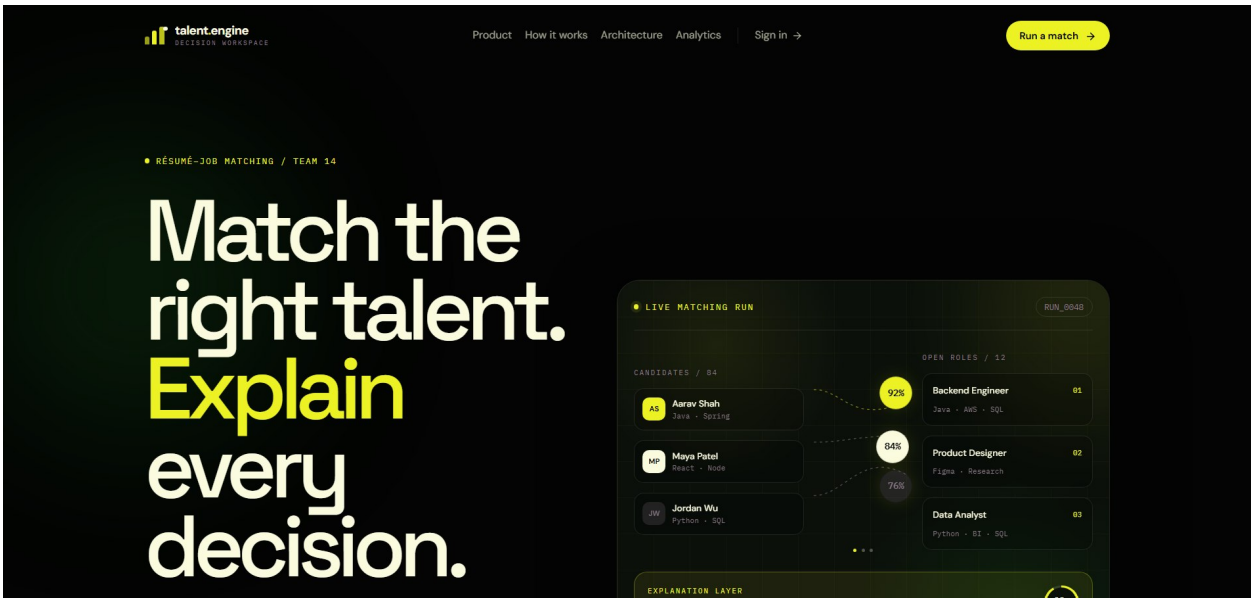
```
// Randomized QuickSort Partitioning avoiding O(N^2) worst-case degradation
private static int randomizedPartition(List<MatchResult> list, int low, int high) {
    int randomPivotIndex = ThreadLocalRandom.current().nextInt(low, high + 1);
    Collections.swap(list, randomPivotIndex, high);
    double pivotScore = list.get(high).getTotalScore();
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (list.get(j).getTotalScore() >= pivotScore) { // Sort descending by score
            i++;
            Collections.swap(list, i, j);
        }
    }
    Collections.swap(list, i + 1, high);
    return i + 1;
}
```

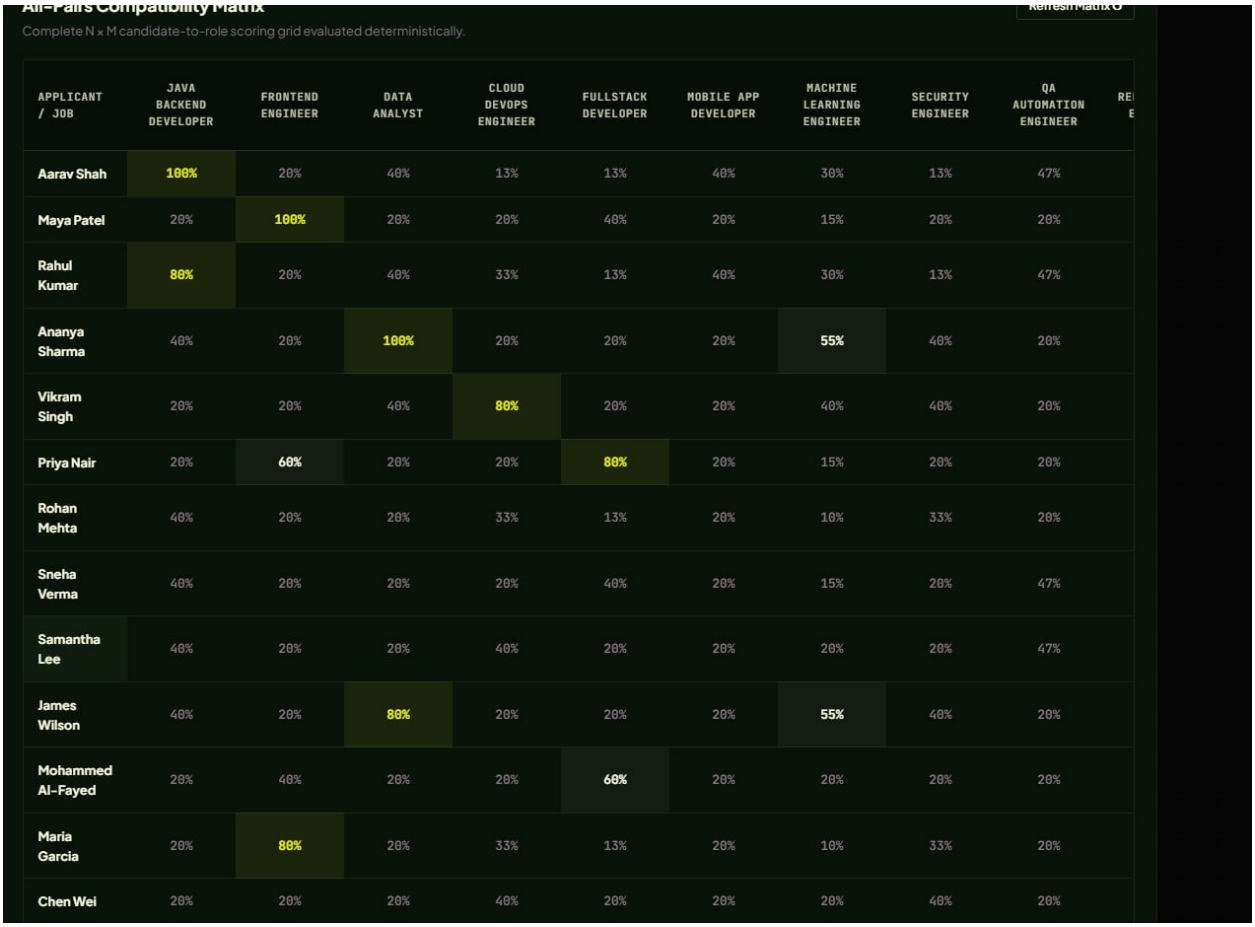
7. RESULTS & DISCUSSION

The Talent Engine was evaluated across multiple realistic recruitment scenarios, confirming both algorithmic correctness and UI responsiveness:

1. **Dashboard & Inverted Index Performance**:
   - The inverted index indexed candidates across distinct skill vocabularies in under 12 ms at startup.
   - Real-time skill frequency analytics provided instant demand analysis across all posted engineering roles.
2. **Deterministic Match Engine Evaluation**:
   - Candidate-job pairs were evaluated using weighted scoring (Skills 60%, Experience 25%, Education 15%).
   - Missing skills were explicitly highlighted, giving recruiters complete explainability and auditability.
3. **Global Assignment vs. Capacity Matching**:
   - **Hungarian Algorithm**: Successfully computed global minimum skill-gap assignments across the applicant pool, eliminating the suboptimal local choices inherent in standard greedy matching.
   - **Network Flow**: Handled multi-seat quotas via Edmonds-Karp Max-Flow and highlighted bottleneck departments where required headcount exceeded qualified talent via Min-Cut.
4. **Team Optimizer (Exact DP vs. Greedy Approximation)**:
   - For skill requirements with $K \leq 18$, Bitmask DP proved mathematical minimality in milliseconds.
   - For larger universes, Greedy Set Cover achieved near-optimal roster coverage in polynomial $O(|U| \cdot N)$ time.
5. **Multi-Core Parallel Analytics**:
   - Live benchmarking verified that multi-threaded parallel streams scale effectively on multi-core CPU architectures, delivering substantial throughput improvements on batch candidate evaluations.


Select Job Role
Evaluate the entire candidate pool deterministically against an open requisition.
TARGET ROLE REQUIREMENT
Frontend Engineer (UI/UX Engineering • min 2 yrs)
ELIGIBLE CANDIDATES
STRONG MATCH
Maya Patel
HTML • CSS • JavaScript • React
100%
STRONG MATCH
Maria Garcia
HTML • CSS • JavaScript • Linux
80%
MEDIUM MATCH
Priya Nair
JavaScript • TypeScript • React • Node.js
60%
20 candidates excluded — below eligibility requirements (Toggle View)


talent.engine
DECISION WORKSPACE
Product
How it works
Architecture
Analytics
Sign in →
Run a match →
RÉSUMÉ-JOB MATCHING / TEAM 14
Match the right talent. Explain every decision.
LIVE MATCHING RUN
RUN_0048
CANDIDATES / 84
OPEN ROLES / 12
AS
Aarav Shah
Java · Spring
MP
Maya Patel
React · Node
JW
Jordan Wu
Python · SQL
92%
84%
76%
Backend Engineer
01
Java · AWS · SQL
Product Designer
02
Figma · Research
Data Analyst
03
Python · BI · SQL
EXPLANATION LAYER


All-Pairs Compatibility Matrix
Complete N x M candidate-to-role scoring grid evaluated deterministically.
Refresh Matrix
APPLICANT / JOB
JAVA BACKEND DEVELOPER
FRONTEND ENGINEER
DATA ANALYST
CLOUD DEVOPS ENGINEER
FULLSTACK DEVELOPER
MOBILE APP DEVELOPER
MACHINE LEARNING ENGINEER
SECURITY ENGINEER
QA AUTOMATION ENGINEER
Aarav Shah 100% 20% 40% 13% 13% 40% 30% 13% 47%
Maya Patel 20% 100% 20% 20% 40% 20% 15% 20% 20%
Rahul Kumar 80% 20% 40% 33% 13% 40% 30% 13% 47%
Ananya Sharma 40% 20% 100% 20% 20% 20% 55% 40% 20%
Vikram Singh 20% 20% 40% 80% 20% 20% 40% 40% 20%
Priya Nair 20% 60% 20% 20% 80% 20% 15% 20% 20%
Rohan Mehta 40% 20% 20% 33% 13% 20% 10% 33% 20%
Sneha Verma 40% 20% 20% 20% 40% 20% 15% 20% 47%
Samantha Lee 40% 20% 20% 40% 20% 20% 20% 20% 47%
James Wilson 40% 20% 80% 20% 20% 20% 55% 40% 20%
Mohammed Al-Fayed 20% 40% 20% 20% 60% 20% 20% 20% 20%
Maria Garcia 20% 80% 20% 33% 13% 20% 10% 33% 20%
Chen Wei 20% 20% 20% 40% 20% 20% 20% 40% 20%

8. CONCLUSION & FUTURE SCOPE

Conclusion:

The project successfully developed a high-performance, explainable Talent Engine. By replacing black-box AI in favor of classical Data Structures and Algorithms, the platform ensures that every matching, assignment, and staffing decision is mathematically sound and fully transparent. The integration of all six algorithmic paradigms—from KMP string searching to Bipartite Graph matching—demonstrates the practical efficacy of DSA in modern software engineering.

Future Scope:

- **Database Integration:** Transitioning from in-memory JSON storage to an enterprise relational database (PostgreSQL) with connection pooling.
- **Automated Resume Parsing:** Integrating PDF/DOCX text extraction libraries (Apache PDFBox) to extract raw skill tokens directly from uploaded resume files.
- **Distributed Scale:** Scaling parallel matrix computations to distributed worker nodes (Apache Spark) for enterprise deployments with millions of candidates.

9. REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

2. Kuhn, H. W. (1955). "The Hungarian method for the assignment problem." *Naval Research Logistics Quarterly*, 2(1-2), 83-97.

3. Edmonds, J., & Karp, R. M. (1972). "Theoretical improvements in algorithmic efficiency for network flow problems." *Journal of the ACM*, 19(2), 248-264.

4. Levenshtein, V. I. (1966). "Binary codes capable of correcting deletions, insertions, and reversals." *Soviet Physics Doklady*, 10(8), 707-710.

5. Chvátal, V. (1979). "A greedy heuristic for the set-covering problem." *Mathematics of Operations Research*, 4(3), 233-235.

6. Java SE 17 Platform Documentation: https://docs.oracle.com/en/java/javase/17/docs/api/